

DJI Matrice 210 Integration Plan

Scarecrow Drone - Bird Detection & Deterrence System

GPS-Free Design

1. Overview

The Problem

Our system needs to control the **DJI Matrice 210** drone from Python · but DJI only provides a C/C++ SDK with no native Python support. Additionally, **GPS is not available**.

The Solution

Use **RosettaDrone** · an Android app on the RC controller that translates DJI protocol into **MAVLink** (an open drone standard), allowing our Python backend to control the drone using **MAVSDK-Python**.

For navigation without GPS, the drone uses a **pre-programmed patrol pattern** (lawnmower/zigzag) defined by area size, speed, and altitude · fully automatic from takeoff, no manual calibration needed. The DJI M210 built-in sensors (downward vision + IMU + barometer) handle stability. If drift proves too large in practice, **ArUco markers** on the ground can be added as a fallback for position correction.

Why MAVSDK-Python (not DroneKit)

DroneKit-Python was the standard library for MAVLink drone control in Python, but it has been **abandoned since 2019**. It crashes on Python 3.10+, doesn't support async (which our FastAPI backend requires), and has no Windows support without manual patches.

MAVSDK-Python is its modern replacement · actively maintained by the Dronecode Foundation (last release Feb 2026), fully supports Python 3.10+, and is natively async, which integrates cleanly with our FastAPI backend.

What We Need

- DJI Matrice 210 drone + remote controller
 - Android phone running RosettaDrone (free, open-source)
 - Phone and PC on the same WiFi network
 - No additional hardware (ArUco markers only if drift correction is needed)
-

2. System Architecture

DJI M210 Drone · (DJI wireless link) · **Remote Controller + Android Phone (RosettaDrone)** · (UDP over WiFi) · **PC (FastAPI Backend)** · (WebSocket + REST) · **React Frontend Dashboard**

The data flows through four stations:

- 1 **DJI M210 Drone** · Flies using its built-in vision sensors and IMU (no GPS). Sends video and telemetry to the remote controller.
- 2 **Remote Controller + Android Phone** · The phone runs the RosettaDrone app, which translates DJI proprietary protocol into MAVLink. It forwards drone telemetry on UDP port 14550 and video on

UDP port 5600 to the PC.

- 3 **PC (FastAPI Backend)** · Runs MAVSDK-Python to send flight commands, YOLO for bird detection, and the chase/scouting logic. This is the brain of the system.
 - 4 **React Frontend Dashboard** · Displays live video, telemetry, and flight controls to the operator via WebSocket and REST API.
-

3. GPS-Free Navigation

Since GPS is unavailable, all navigation is **velocity-based** · no coordinate commands are used.

- **Stability:** The DJI M210 has built-in downward vision sensors and IMU that handle hover and stability without GPS automatically. We don't need to implement this ourselves.
- **Position tracking:** Dead reckoning · the system tracks where the drone is based on the velocity commands it sends and the time elapsed. The IMU provides heading. DJI's onboard vision sensors also help maintain position during hover.
- **Scouting path:** The operator configures the patrol area size (e.g., 30m × 15m), altitude, and speed from the dashboard. The system generates a lawnmower/zigzag pattern automatically · no manual flying or calibration needed. The pattern is converted into a sequence of velocity commands and turns.
- **Return home:** When the patrol is complete or the operator recalls the drone, it reverses the velocity sequence back to the starting point. The operator can also take manual control at any time.

Fallback: ArUco Markers

If dead reckoning drift is too large in testing, ArUco markers (printed patterns on A4 paper) can be placed on the ground as position correction checkpoints. OpenCV detects them instantly and gives exact position · no extra hardware, just printed paper.

4. Autonomous Flight Cycle

The drone operates in a continuous loop:

TAKEOFF · SCOUT · BIRD DETECTED · CHASE · BIRD GONE · RESUME SCOUTING · ... ·
RECALL or PATH COMPLETE · RETURN · LAND

- **Scout** · The drone flies the auto-generated patrol pattern while YOLO scans every video frame for pigeons.
 - **Chase** · When a pigeon is detected, scouting pauses. The drone flies toward the pigeon using velocity commands based on where the bird appears in the camera frame (left/right/forward).
 - **Resume** · Once no pigeon is detected for 5 seconds, the drone resumes scouting from where it left off.
 - **Return** · When the operator recalls the drone or the patrol pattern is complete, the drone reverses the velocity sequence back to the launch point and lands.
-

5. Key MAVSDK Commands

All flight control uses velocity commands · no GPS-dependent functions:

- `Action.arm() / Action.takeoff()` · Arms motors and takes off to a set altitude (barometer-based, no GPS needed).
- `Offboard.set_velocity_body()` · Sends velocity vectors for all movement: scouting and

- chasing. This is the primary flight control method.
- `Action.land()` · Lands at the current position.
- Telemetry **streams** · Attitude (roll/pitch/yaw), altitude, battery level, flight mode.

GPS-dependent commands like `goto_location()` and `return_to_launch()` are **not used**.

6. New Dependencies

- `mavsdk` · MAVLink drone control library for Python (async). This is the only new dependency.
 - `opencv-python` · Already in the project for YOLO. Would also be used for ArUco marker detection if the fallback is needed.
-

7. Risks & Mitigations

- **Dead reckoning drift over long flights** · The drone's path will become less accurate over time without position corrections. For short patrols (5-10 minutes) this is acceptable. If not, ArUco markers can be added as correction checkpoints.
- **RosettaDrone doesn't support Offboard velocity mode** · This would prevent autonomous flight. Should be tested early. Fallback: use RC channel override commands instead.
- **MAVSDK won't arm without GPS lock** · By default, the flight controller requires GPS. This can be configured to allow GPS-less arming through flight controller parameters.
- **M210 discontinued (EOL Jan 2026)** · No DJI support for repairs. Acceptable for an academic project.